

A NOVEL PHISHING CLASSIFICATION BASED ON URL FEATURES

Mahmoud Khonji, Andrew Jones, Youssef Iraqi

Khalifa University of Science, Technology and Research (KUSTAR)
{mkhonji, andrew.jones, youssef.iraqi}@kustar.ac.ae

ABSTRACT

Phishing is the process of illicitly obtaining data through social engineering via electronic communication channels. As reported by the Anti-Phishing Working Group (APWG)¹, Phishing attacks are growing in volume and sophistication. As a result, the need to improve Phishing detection methods increases. We introduce a simple and novel Phishing classification feature that aims toward supplementing existing classifiers by detecting a subset of Phishing attacks.

Index Terms— Phishing, classification, URL

1. INTRODUCTION

We define Phishing as a form of crime that unlawfully obtains data from victims for an attacker's benefit over an electronic communication channel through *Social Engineering*. Social engineering is a mechanism that leads the victim into performing certain actions, such as submitting personal information directly to the Phisher, or executing malicious software that would in turn submit the data to the Phisher without victims knowledge.

Several Phishing solutions exist to describe Phishing attack features, which would in turn assist classifiers. However, as of our knowledge, none of them address the feature we propose in this paper.

It is in reality difficult to find a single heuristic that matches all, or most of, Phishing attacks with low false positives. In several proposed solutions, such as [1], combinations of multiple heuristic features were used to achieve results closer to the ideal theoretical boundary.

In this paper, we propose a novel heuristic classification feature, that targets a subset of Phishing attacks based on certain URL features. Our goal is not to detect all Phishing attacks, but to provide a new heuristic feature that supplements other solutions; ultimately increasing the overall classification efficiency.

The rest of the paper is organized as follows. Background and related work is presented in Section 2. The scope of study is described in Section 3. Section 4 and 5 described the proposed solutions and performance evaluation respectively. Finally, Section 6 concludes the paper.

2. BACKGROUND AND RELATED WORK

Phishing solutions can be classified into categories: Blacklists, heuristics, machine learning, trusted communication, visual similarity and hybrid.

Blacklists are regularly maintained lists of illicit sources. As studied in [2], blacklists are found to be ineffective against zero-hour attacks, and were able to detect only 20% of them. The study also shows that 47-83% of Phishing URL were blacklisted after 12 hours. This delay is a significant issue as 63% of Phishing campaigns end within the first 2 hours.

Heuristic systems classify based on certain behaviors that are generally seen to work (but not guaranteed). Evaluation in [2] shows that heuristics are able to constantly detect two to three times more Phishing attacks at zero-hour than blacklists. However, heuristics caused a higher false positive, while our proposal has low false positives.

In [3], Native Bayes filters were used to fight Phishing, and were able to achieve 100% true positive against Phishing, however an extensive whitelist had to be used to alleviate false positives. Our proposal does not require maintaining a whitelist.

Trusted messaging can make legitimate sources unique by authentication. Examples are DKIM², and SPF³. As a positive step in [4], PayPal ensures that all its outbound emails are DKIM signed and that Gmail will discard any unsigned or unverified PayPal email. However, many businesses still do not implement DKIM and thus protection is limited between certain groups of organizations. Also, many users ignore information provided by toolbars/address bars [5].

Hybrid classifiers such as Netcraft toolbar and Phishing honey-pots such as [6] combine multiple features to detect Phishing attacks (e.g. heuristics and blacklists).

3. SCOPE OF STUDY

Since Phishing attacks aim to deceive the victim into believing that an illicit site is legitimate, a subset of Phishing attacks forge their URL in a form to contain the legitimate domain name. Table 1 shows samples of Phishing URL downloaded through the PhishTank API⁴.

Thanks to Buhooth for funding.

¹<http://antiphishing.org>.

²DomainKey Identified Mail

³Sender Policy Framework

⁴<http://www.phishtank.com>

Table 1. Examples of Phishing websites where the victim domain is included in the URL.

Example Phish URL	Comment
http://login.paypal.com.nedgy.com	Victim domain as a subdomain
http://12.www.edn.cn/sell.ebay.com	Victim domain in URL path

However, we can not heuristically classify any string occurrence in a URL that matches public DNS domain names as Phishing. Table 2 shows an example.

Table 2. Example of a suspect URL

Example Suspect URL	Comment
http://paypal.com/login.pl	.pl is both a Top Level Domain (TLD) and a common extension for Perl scripts.
http://mail.google.com/a/example.org	Google Apps providing mail service to <i>example.org</i>

The scope of our classification mechanism is to classify a subset of Phishing attacks that include the victim domain name in the URL without the need for using blacklists to minimize maintenance overheads.

Based on our analysis of 60,000 URL collected from PhishTank within 20 days, and 160,000 URLs from 8 volunteers between 3 Apr 2010 and 3 Jul 2010, an average of 13% of total Phishing attacks included victim domain names in the Phishing URL.

In comparison to other existing classification heuristics, our analysis indicated that 4% of Phishing attacks used IP addresses instead of hostnames. This is a relatively common classification heuristic proposed as a part of several solutions, such as [1]. Figure 1 compares the detection scope of an existing classification method against our classification proposal.

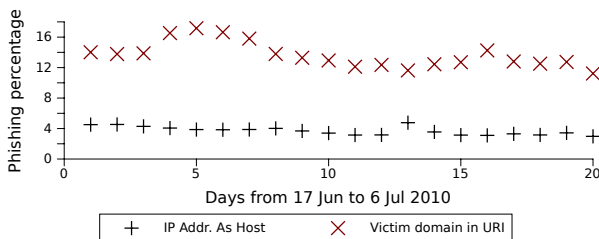


Figure 1. Daily analysis of Phishing URL features.

4. PROPOSED SOLUTION

URLs that fall under our detection scope (examples shown in Tables 1) are probably Phishing URLs. However, in order to reduce false positives (e.g. innocent URLs shown in Table 2), we calculate ranks of all domains that exist in a given URL (e.g. http://paypal.com.phish.com/ has one domain name, being *paypal.com.phish.com*, however it lexically contains the name of another domain, being *paypal.com*). If ranks of heuristically detected domain names within a URL are greater than the rank of the actual domain name by a certain offset, and the actual domain name's rank falls below certain thresholds, then our heuristic feature returns a positive value (e.g. classified as Phish).

Although it is possible for Phishers to increase the ranks of their domains, most Phishing attacks do not live long enough to have their rank raised. Ranks are, practically, effective methods to reduce false positive, and are currently used in number or anti-Phishing proposals, such as [1]. As studied in [2], 63% of Phishing campaigns live for 2 hours only.

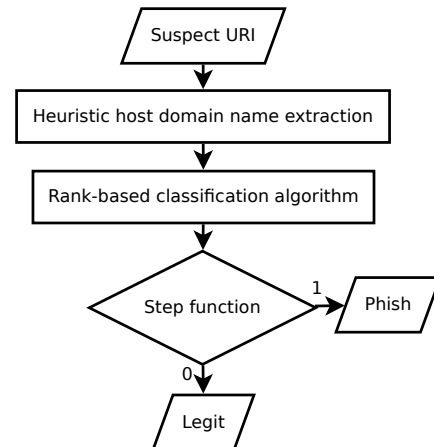


Figure 2. The overall process of the proposed heuristic classifier.

Figure 2 shows the overall process, which will be explained in subsequent sections. Section 4.1 explains the algorithm of our heuristic feature, Section 4.2 explains how we heuristically extract victim domain names within a given URL, Section 4.3 explains the mechanism we used to calculate domain ranks, and Section 5.2 explains how we selected algorithm parameter values (i.e. rank offsets and thresholds).

4.1. Classification Algorithm

Algorithm 1 presents our heuristic feature. R_h is domain h 's rank, R_v is domain v 's rank, and $rank()$ is a function that returns the rank of a given domain. R_o is rank offset that enables the flexibility of adjusting the difference between R_h and R_v . R_m is a minimum rank threshold — domains falling above R_m will be protected from being classified as Phish.

Algorithm 1 Rank-based heuristic classification.

```

 $R_h \leftarrow \text{rank}(h)$ 
 $R_v \leftarrow \text{rank}(v)$ 
if  $((R_v - R_h) > R_o)$  and  $(R_h < R_m)$  then
    return(1)
else
    return(0)
end if

```

4.2. Host and Victim Domain Extraction

There is no way to accurately extract victim domain names from a URL as they are positioned as subdomains, paths, queries or fragments.

Thus we extract domain names by lexically analyzing a given URL, and using a pre-defined list of Top Level Domains (TLD) to identify any occurrence of a string matching a TLD. In case a TLD is matched in the URL, any preceding string will be heuristically assumed to assemble a domain name. We limited strings to 0-9, a-z (case insensitive) and “-” to exclude non-domain characters as defined in RFC1035.

We introduce two modes to extract host domain names: strict, and loose. The difference is related to how host’s domain is extracted. See table 3.

Table 3. Strict and loose mode examples.

Complete host name	Mode	Returned host name
subdom.paypal.com	Strict	subdom.paypal.com
subdom.paypal.com	Loose	paypal.com

4.3. Domain Rank Extraction

The domain ranking algorithm is done via *citation counting*, where the rank of document $r(A)$ equals number of other documents n that link back to document A . See Equation 1.

$$r(A) = n(A) \quad (1)$$

The returned domain rank is dependent on the domain name extraction mode specified in Section 4.2. In strict mode, the rank is calculated against the full hostname, including any subdomains. With loose mode, the rank is calculated against the domain name excluding any subdomains. The differences between the two tests were noticeable as loose mode produced higher ranks than strict mode.

For example, “http://paypal.com/login.pl” is a legitimate URL where the hostname is “paypal.com” and the heuristically detected domain is “login.pl” (.pl is Poland’s TLD). We used a search engine to execute our queries “paypal.com” -site:paypal.com and “login.pl” -site:login.pl. We consider the quantity of returned entries as domain ranks. The ranks were 8,520,077 and 333,028 respectively.

5. PERFORMANCE EVALUATION**5.1. Implementation Details**

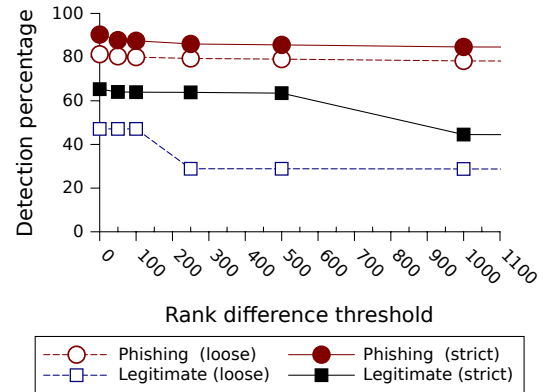
The primary objective of our implementation is testing the classification accuracy of our heuristic feature, and is not intended to be production grade at this stage. However, integrating it with existing MTAs, such as Postfix⁵, or SPAM filters, such as SpamAssassin⁶, is a relatively simple task.

We implemented our heuristic feature in Perl, and run the tests in a Gentoo Linux 2.6.34 workstation with total 4GB of RAM, and Intel Core 2 Duo E8400 processor with each core clocked at 3.00GHz. The total available Internet bandwidth in our case is 4Mbps with varying throughput depending on access time. Our choice of Perl is primarily to enable easier integration with SpamAssassin should we decide to. This is due to the fact that SpamAssassin and most of its tests are written in Perl. However, our use of Gentoo Linux is mainly due to our personal preference.

We used web search engines to count number of backlinks for extracting domain ranks introduced in Section 4.3. The search query format we used is: “example.com” -site:example.com.

5.2. Algorithm Parameter Values

Parameters R_o and R_m are primarily used to reduce false positives with URLs that fall under our detection scope. We selected the optimum value of each parameter by analyzing Phishing and legitimate samples independent of each other. Based on Figures 3 and 4, R_o ’s and R_m ’s optimum values toward the highest true positive and lowest false positive are 250 and 400 respectively for *loose* mode, and 1,000 and 50 respectively for *strict* mode. *Loose* and *strict* modes are explained later in Section 4.2.

**Figure 3.** Analysis results of optimum R_o value.**5.3. Evaluation of the Proposed Heuristic Feature**

Similar to many systems that rely on resources accessed through the Internet (e.g. DNSBLs), our proposed solution is effected by the available Internet throughput. Al-

⁵http://postfix.org⁶http://spamassassin.apache.org

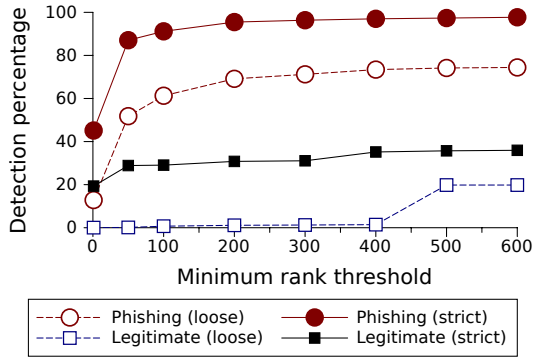


Figure 4. Analysis results of optimum R_m value.

though increasing the speed is not our concern at the moment of writing this paper, we cached domain ranks to run tests against thousands of URLs with relatively acceptable delays. Our rank cache consists of files stored on a local SATA HDD, which is indeed not ideal, however it did make the tests run quick enough for our experimental use.

Although the implementation is not optimized toward better speed, it takes approximately 1.886 seconds to process 2,000 URLs when domain ranks are cached, and 5 minutes and 57.88 seconds when no domain ranks are cached. We introduced a 5sec sleep timer between domain rank search queries for compliance with search engine terms of use, which further raised the delay for uncached ranks. The delay caused by search queries can be avoided if a different service is used to supply domain ranks, such as Alexa⁷'s ranked list of top 1 million domains. We did not investigate other ranking methods due to time constraints and due to lack of sufficient relevance to the main subject.

We evaluated the classification accuracy of our proposed heuristic feature against URLs obtained from Phish-Tank and daily network traffic of 8 volunteers. The URL corpus is composed of more than 60,000 Phishing URLs (with 8,000 unique), and over 160,000s legitimate URLs (with 13,271 unique). URLs were collected from 24 April 2010 to 3 July 2010. We also used optimum R_o and R_m values studied in Section 5.2.

We ran our heuristic classifier against Phishing URLs, and the percentage of correct classification (true positives) within our scope was 62.9% and 83.31% for loose and strict modes respectively. We also ran our classifier against legitimate URLs and percentage of misclassification (false positives) was 1.39% and 27.34% for loose and strict modes respectively. Although strict mode detected more Phishing URL, loose mode had significantly lower false positives and achieved better ratio in overall.

On the other hand, the existence of IP addresses instead of domain names — a relatively common heuristic feature seen in many proposed solutions, such as [1] — yielded 3.66% true positives and 0.11% false positives with the same corpus of Phishing and legitimate URLs.

6. CONCLUSION

This paper proposed a novel heuristic Phishing feature found in a subset of Phishing URLs that aims to supplement existing heuristic solutions.

Use of this heuristic classification system might not be efficient on HTTP clients due to the delay with HTTP search queries, and therefore we suggest implementing it on a mail server where email content is checked passively without imposing a delay on client applications. Our next step is integrating our heuristic feature with an existing heuristic system, such as SpamAssassin.

Our implementation used search engines to calculate backlinks, which caused delays. Using an alternative mechanism might lead to substantial speed increase.

7. REFERENCES

- [1] Colin Whittaker, Brian Ryner, and Marria Nazif, "Large-scale automatic classification of phishing pages," <http://research.google.com/pubs/pub35580.html>, Accessed July 2010.
- [2] Steve Sheng, Brad Wardman, Gary Warner, Lorie Faith Cranor, Jason Hong, and Chengshan Zhang, "An empirical analysis of phishing blacklists," <http://ceas.cc/2009/papers/ceas2009-paper-32.pdf>, 2009, Accessed July 2010.
- [3] Peter Likarish, Don Dunbar, and Thomas E. Hansen, "B-apt: Bayesian anti-phishing toolbar," *Communications, 2008. ICC '08. IEEE International Conference on*, pp. 1745 – 1749, May 2008.
- [4] "Fighting phishing with eBay and PayPal - official gmail blog," <http://gmailblog.blogspot.com/2008/07/fighting-phishing-with-ebay-and-paypal.html>, Accessed July 2010.
- [5] Min Wu, Robert C. Miller, and Simson L. Garfinkel, "Do security toolbars actually prevent phishing attacks?," in *CHI '06: Proceedings of the SIGCHI conference on Human Factors in computing systems*, New York, NY, USA, 2006, pp. 601–610, ACM.
- [6] Erwin P. Rathgeb and Dirk Hoffstadt, "The e-mail honeypot system concept, implementation and field test results," in *ICDS '08: Proceedings of the Second International Conference on Digital Society*, Washington, DC, USA, 2008, pp. 1–6, IEEE Computer Society.

⁷<http://www.alexa.com>.